



1

7

840 sp

عملي

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

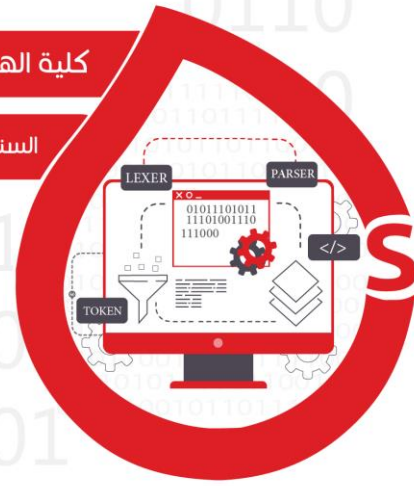
مراجعة لمفاهيم lexer والparser

م.وسيم البزرة، آية شحادة

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 13/4/2025

المترجمات



- بسم الله نبدأ.... مع أولى خطواتنا مع مادة الكومبايلر، نرجو أن تكون هذه الرحلة المعرفية خفيفة وممتعة، تحمل في طياتها الفائدة لكم.
- تتكون المادة من ثلاث محاضرات فقط
- تقسيم العلامات: 20 مقابلة مرحلية أولى - 20 مقابلة مرحلية ثانية - 60 مقابلة نهائية (علامة تراكمية ويمكن فردية)

أول محاضرتين مراجعة للفصل الماضي

- مراحل العمل: القواعد $\leftarrow$ AST $\leftarrow$ visitor $\leftarrow$ symbol table $\leftarrow$ semantic error $\leftarrow$ code generation

أولاً: مرحلة lexical analyst

- معنى lexical: هو الشيء الثابت الموجود لدينا مثل: الأقواس، النقطة، العمليات الحسابية، التساوي، كلمة type بلغة الجافا سكربت أي بشكل عام هي key words وتدعى أيضاً Token

مثال:

```
main(){
  int a, b;
  a = 10;
  return a;}
```

tokens : الفاصلة، الأقواس، return, =, int



ملاحظة: شرح بعض الرموز في قواعد التحليل النحوي

- إشارة الاستفهام؟: معناها أن العنصر السابق ممكن أن يظهر مرة واحدة أو لا يظهر نهائياً (اختياري)
- النجمة *: معناها أن العنصر يتكرر أي عدد من المرات (صفر أو أكثر)
- Skip: معناها تجاهل هذا العنصر بالكامل غالباً تستخدم مع الفراغات والتعليقات
- Whitespaces: أي الفراغات (ws: [\t\r\n] + $\rightarrow$ skip) newlines-tab-space

ما هو الـ mode:

- هو عبارة عن scope خاص من القواعد يقوم بصنع مكس تكون عناصره منفصلة عن بقية العناصر الأخرى الموجودة بالملف
- كيف تعمل الـ Modes ؟

يبدأ الـ Lexer بوضع افتراضي (عادة يسمى DEFAULT)
عند مواجهة محفز معين (مثل علامة اقتباس لفتح نص)، ينتقل إلى وضع جديد
في الوضع الجديد، يتم تطبيق قواعد مختلفة للتوكنات
عند مواجهة محفز إنهاء (مثل علامة اقتباس لإغلاق نص)، يعود إلى الوضع السابق الافتراضي

مثال اريسيزي :

```
lexer grammar ExampleLexer;
```

```
// الأوضاع
```

```
mode DEFAULT_MODE;
```

```
OPEN_COMMENT : '/*' -> pushMode(COMMENT_MODE);
```

```
OPEN_STRING : '"' -> pushMode(String_Mode);
```

```
ID : [a-zA-Z]+;
```

```
NUMBER : [0-9]+;
```

```
WS : [ \t\r\n]+ -> skip;
```

```
mode COMMENT_MODE;
```

```
CLOSE_COMMENT : '*/' -> popMode;
```

```
mode String_Mode;
```

```
CLOSE_STRING : '"' -> popMode;
```



1. الوضع الافتراضي: (DEFAULT_MODE)

OPEN_COMMENT: عند رؤية /* ننتقل إلى وضع التعليقات

OPEN_STRING: عند رؤية " ننتقل إلى وضع النصوص

ID, NUMBER: نتعرف على المعرفات والأرقام

WS: نتجاهل المسافات البيضاء

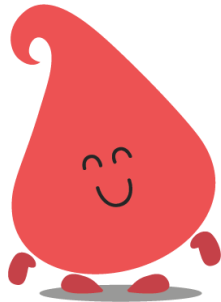
2. وضع التعليق: (COMMENT_MODE)

CLOSE_COMMENT: عند رؤية */ نعود للوضع السابق

3. وضع النص: (String_Mode)

CLOSE_STRING: عند رؤية " نعود للوضع السابق

مثال على ما سبق :



```
print("Hello /* not a comment */")
```

يبدأ ب: DEFAULT_MODE

يتعرف على print ك ID

يتعرف على " فينتقل ل: STRING_MODE

يلتقط Hello /* not a comment */ كسلسلة واحدة

لا يهتم ب /* لأنه في وضع النصوص!

يتعرف على " الأخرى فتعيده ل: DEFAULT_MODE

من أحد المشاكل التي تواجه الطلاب وجود for ضمن القواعد لا يتم قراءتها كقاعدة وإنما متحول لماذا؟؟

مثال:

```
IDENTIFIER: [a-zA-Z_][a-zA-Z0-9_]*;
FOR: 'for';
```

▪ كيف ستتم القراءة؟

أولا يقرأ الid وعندما يصل لfor يعتبرها identifier

▪ الحل!

وضع جميع الkeyWords قبل القواعد بالتالي الترتيب جداً هام في ملف الlexer

ممکن بحالة أن يكون لدينا متحول ولا أريد أن يراه الparser يمكن وضعه بالlexical لكن في هذه الحالة يصبح اسمه fragment

مثلاً: لدينا id رقم من الصفر لتسعة لمنع القواعد أن تراه اعرفه بالlexical كfragment
\\Fragment num

```
ID : ID_LETTER (ID_LETTER | DIGIT)*;
fragment ID_LETTER : 'a'..'z' | 'A'..'Z' | '_';
fragment DIGIT : '0'..'9';
```

التعليمات داخل الlexer يجب أن تكون capital letter أي أحرف كبيرة

لماذا نضع *lexer grammar MyLexer* في بداية ملف الlexical ؟
لكي نخب ANTLR أن هذا الملف مخصص فقط لتعريف قواعد الlexer

ثانياً: مرحلة الـ parser

هو أداة للتعرف على تسلسل الرموز tokens أي هنا يتم كتابة القواعد ويتم مقارنة كل شيء بالـ parser مع المكتوب ضمن الـ lexer

مثال:

```
int a, b;
```

سوف يبدأ بقراءة الـ int التي يجدها بالـ lexer ثم الـ a تم تعريفها حالياً كمتحول ثم الفاصلة موجودة بالـ lexer أيضاً ثم الـ b كمتحول وأخيراً الفاصلة المنقوطة ; أيضاً معرفة بالـ lexer

■ يجب فصل الـ lexical عن الـ parser (بالتالي دمجهم مرفوض كما قال المعيد)

ملاحظات:

- القواعد الطويلة ممنوعة كلياً يجب أن تكون منظمة
- في بناء (Parsers) وتنفيذ Visitors، عندما نعالج جملة (statement)، ممكن يكون عندنا أنواع متعددة: ForLoop- IfStatement-VariableDeclaration - ExpressionStatement... الخ.
- إذا لم يكن هناك تمييز واضح، ممكن أن يحصل خلط، والـ visitor يحاول أن يزور نوع خطأ من العقد ويظهر Exception

مثلاً:

```
statement
: variableDeclaration
| ifStatement
| expressionStatement
;
```

عندما يزور الـ visitor العقدة statement، يجب أن يعرف نوعها بدقة، لكي يذهب للفرع الصحيح.

■ الحل: استخدام "label" في القواعد

ما هو الـ label: يتم استخدامه للتحكم في الـ flow والتوجيه ونستخدمه لتحديد العمليات المرتبطة بنقاط معينة بالكود لدينا.. مثل الحلقات والشروط وغيرها

```
statement
: decl=variableDeclaration      #VarDeclStatement
| cond=ifStatement              #IfStmt
| expr=expressionStatement      #ExprStmt
;
```



ANTLR يولد واجهة اسمها YourLanguageVisitor، ويضيف فيها :

```
T visitVarDeclStatement(YourParser.VarDeclStatementContext ctx);
T visitIfStmt(YourParser.IfStmtContext ctx);
T visitExprStmt(YourParser.ExprStmtContext ctx);
```

مثال:

```
main(){
isTrue = true
if(isTrue){ true label1;}
else{ false label1;}
true label1: std:.....return;
false label1: std:.....return;
```

اولاً نحدد ماهي قيمة المتغير ثم
نفحص القيمة داخل الشرط اذا
محقة ننتقل للعلاقة الخاصة بها
true label1 و اذا خاطئة ننتقل
للعلاقة false label1 ويعطي خرج
قيمة خاطئة

ما هي الـ node وما فائدتها؟

تأشر على العناصر الضرورية (الأساسية) التي تكون جزء من شجرة الـ parser أو من شجرة
الـ abstract حيث تكون هي نقاط البيانات بالشجرة وفيها المعلومات الأساسية وتمثل البيانات
بشكل هرمي للسهولة بمرحلة الترجمة

مثال:

جذر الشجرة Program node

statement node

Output Node

Constant Node يضم الأشياء المطلوبة بالخرج



ما دمت نسعى، فأنت على الطريق
الصحيح....

مثال بسيط للتوضيح:

4. ملف الlexer

هذا السطر يُعرف أن هذا الملف هو Lexer grammar في ANTLR

```
lexer grammar ExprLexer;
//WhiteSpace
WS: [ \t\r\n]+ -> skip;
//Operators
PLUS: '+';
//Identifiers
IDENTIFIER: [a-zA-Z_][a-zA-Z0-9_]*;
//Comments
LINE_COMMENT: '//' ~[\r\n]* -> skip;
BLOCK_COMMENT: '/*' .*? '*/' -> skip;
```



القاعدة IDENTIFIER تتعامل مع المتغيرات أو المعرفات.

[a-zA-Z_] يعني أن أول حرف يجب أن يكون حرفاً صغيراً أو كبيراً أو شرطة سفلية.
 [a-zA-Z0-9_]* يعني أن الحروف التالية يمكن أن تكون حروفاً صغيرة أو كبيرة أو أرقاماً أو شرطة سفلية، ويمكن أن تحتوي على أي عدد من هذه الأحرف (صفر أو أكثر).

القاعدة LINE_COMMENT وتتعامل مع التعليقات في سطر واحد.

- '//': تعني أن التعليق يبدأ برمز //.
- ~[\r\n]*: تعني أي شيء يمكن أن يتبع // إلا إذا كان هناك سطر جديد \r أو \n، وبالتالي يتم تجاهل كل شيء حتى نهاية السطر.
- > skip: يعني أن التعليقات يتم تجاهلها ولا تُعتبر توكنات.

القاعدة BLOCK_COMMENT وتتعامل مع التعليقات المتعددة الأسطر.

- '/*': تعني بداية التعليق المتعدد الأسطر.
- ?*': تعني أي شيء (أي عدد من الحروف) حتى نصل إلى النهاية.
- '*/': تعني نهاية التعليق المتعدد الأسطر.
- > skip: يعني أن التعليقات المتعددة الأسطر تُتجاهل أيضاً.



Keep going, you're getting there...

5. ملف الـ parser

هذا السطر يُعرف أن هذا الملف هو Parser grammar في ANTLR `ExprParser; parser grammar`

`options { tokenVocab=ExprLexer; }`

`expression: variable (PLUS variable)+;`

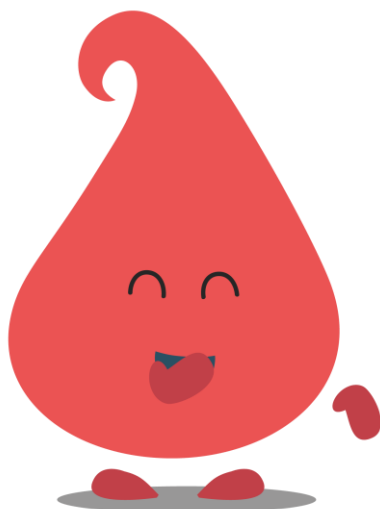
`variable: IDENTIFIER;`

- السطر الثاني يحدد الخيار `tokenVocab=ExprLexer` ويقول لا `Parser` أن يستخدم التوكنات التي تم تعريفها في `Lexer` الذي في الملف `ExprLexer.g4` (مثل `IDENTIFIER`, `PLUS`.. إلخ)

- القاعدة `expression` هي القاعدة الأساسية لتحليل تعبيرات جمع المتغيرات.
- `variable` يشير إلى أن تعبير `expression` يمكن أن يبدأ بمتغير (سيتم تحديده بواسطة القاعدة التالية `(variable (PLUS variable)+)` تعني أنه يجب أن يكون هناك على الأقل تكرار واحد لعملية جمع بين المتغيرات.
- مثلاً: `a + b` أو `a + b + c`
- لكن إذا كان المدخل فقط `a` بدون جمع مع أي متغير آخر، فسيُعتبر غير صالح لأننا نحتاج على الأقل إلى `a + b` أو أكثر.

القاعدة `variable` هي تحدد ما هو المتغير.

`IDENTIFIER`: تعني أن المتغير يجب أن يكون من نوع `IDENTIFIER` الذي تم تعريفه في `Lexer`، والذي يشير إلى المتغيرات (مثل `a, b, myVar`, إلخ).



THE_END_SEE_U_SOON